Aula 04 — Estruturas condicionais

Fazendo o programa tomar decisões com if e else

Dieimes Nunes

Retomando a Aula 03

Na aula anterior, trabalhamos com:

- operadores aritméticos;
- expressões matemáticas;
- precedência de operadores;
- operadores relacionais;
- operadores lógicos;
- resultados `True` e `False`;
- testes de mesa.

Exemplos que já vimos:

```
media >= 7
idade >= 18
senha == "1234"
total > 100
```

Hoje vamos usar essas comparações para fazer o programa escolher caminhos diferentes.

O problema da aula

Imagine um sistema escolar simples.

O programa recebe duas notas, calcula a média e precisa informar:

- se o aluno foi aprovado;
- ou se o aluno foi reprovado.

Na Aula 03, conseguiríamos mostrar apenas isto:

```
print(media >= 7)
```

O resultado seria:

```
True
```

ou

```
False
```

Mas isso ainda não é uma resposta amigável para o usuário.

Como queremos que o programa responda?

Em vez de mostrar apenas `True` ou `False`, queremos algo assim:

```
Média: 8.00
Situação: Aprovado
```

ou:

```
Média: 5.50
Situação: Reprovado
```

Para isso, usamos uma **estrutura condicional**.

A estrutura condicional permite que o programa tome uma decisão com base em uma condição.

Decisão no cotidiano

Tomamos decisões o tempo todo.

Exemplo:

```
Se estiver chovendo:
    levar guarda-chuva
Senão:
    sair normalmente
```

Outro exemplo:

```
Se a média for maior ou igual a 7:
    aluno aprovado
Senão:
    aluno reprovado
```

A programação transforma esse raciocínio em código.

Atividade 1 — Decisões do cotidiano

No caderno, escreva três situações usando a estrutura:

```
Se condição:
    ação
Senão:
    outra ação
```

Exemplos de situações:

- idade para entrada em um evento;
- saldo suficiente para uma compra;
- senha correta para acessar um sistema;
- temperatura alta ou baixa;
- produto com ou sem desconto.

Depois escolha uma situação e explique para um colega.

Estrutura condicional simples

A estrutura condicional mais comum em Python usa o comando `if`.

```
idade = 18

if idade >= 18:
    print("Maior de idade")
```

Observe:

- `if` significa **se**;
- depois do `if`, colocamos uma condição;
- a linha termina com dois-pontos `:`;
- o comando que pertence ao `if` fica indentado.

O que é indentação?

Indentação é o espaçamento no início da linha.

Em Python, a indentação define quais comandos pertencem a uma estrutura.

Correto:

```
idade = 18

if idade >= 18:
    print("Maior de idade")
```

Incorreto:

```
idade = 18

if idade >= 18:
print("Maior de idade")
```

No Python, a indentação não é apenas estética. Ela faz parte da linguagem.

Entendendo o fluxo do if

No exemplo:

```
idade = 16

if idade >= 18:
    print("Maior de idade")

print("Fim do programa")
```

A condição `idade >= 18` será avaliada.

Como `16 >= 18` é falso, o `print("Maior de idade")` não será executado.

Mas o `print("Fim do programa")` será executado normalmente, pois ele está fora do `if`.

Atividade 2 — Primeiro if

Crie um arquivo chamado:

`idade_if.py`

Digite o código:

```
idade = int(input("Digite sua idade: "))

if idade >= 18:
    print("Você é maior de idade.")

print("Programa finalizado.")
```

Teste com:

- idade 20;
- idade 15.

Depois responda no caderno:

1. O que aconteceu quando a idade foi 20?
2. O que aconteceu quando a idade foi 15?
3. Qual linha sempre apareceu?

Estrutura condicional composta

Em muitos casos, queremos executar uma ação se a condição for verdadeira e outra se ela for falsa.

Para isso, usamos `if` e `else`.

```
idade = int(input("Digite sua idade: "))

if idade >= 18:
    print("Você é maior de idade.")
else:
    print("Você é menor de idade.")
```

O `else` significa **senão**.

Ele será executado quando a condição do `if` for falsa.

Comparando if simples e if/else

Com `if` simples:

```
if idade >= 18:
    print("Maior de idade")
```

O programa só faz algo quando a condição é verdadeira.

Com `if/else`:

```
if idade >= 18:
    print("Maior de idade")
else:
    print("Menor de idade")
```

O programa sempre escolhe um dos dois caminhos.

Programa: aprovação do aluno

```
nota1 = float(input("Digite a primeira nota: "))
nota2 = float(input("Digite a segunda nota: "))

media = (nota1 + nota2) / 2

print(f"Média: {media:.2f}")

if media >= 7:
    print("Situação: Aprovado")
else:
    print("Situação: Reprovado")
```

A condição principal é:

```
media >= 7
```

Se for verdadeira, o aluno é aprovado.

Se for falsa, o aluno é reprovado.

Atividade 3 — Teste de mesa

Antes de executar no computador, faça o teste de mesa.

```
nota1 = 8
nota2 = 6
media = (nota1 + nota2) / 2

if media >= 7:
    situacao = "Aprovado"
else:
    situacao = "Reprovado"
```

Complete a tabela:

Variável	Valor
`nota1`	
`nota2`	

Variável	Valor
`media`	
`media >= 7`	
`situacao`	

Depois teste no Python.

Atividade 4 — Sistema de notas

Crie um arquivo chamado:

`media_if_else.py`

O programa deve:

1. pedir o nome do aluno;
2. pedir duas notas;
3. calcular a média;
4. mostrar a média formatada;
5. mostrar se o aluno foi aprovado ou reprovado.

Exemplo de saída:

```
Aluno: Ana
Média: 8.50
Situação: Aprovado
```

Depois teste com pelo menos três alunos diferentes.

Condições com texto

Também podemos comparar textos.

Exemplo:

```
senha = input("Digite a senha: ")

if senha == "1234":
    print("Acesso permitido")
else:
    print("Senha incorreta")
```

A comparação é feita com ==.

Cuidado:

```
senha = "1234"
```

é atribuição.

```
senha == "1234"
```

é comparação.

Letras maiúsculas e minúsculas

Em Python, textos são sensíveis a maiúsculas e minúsculas.

```
print("Ana" == "Ana")
print("Ana" == "ana")
```

Resultado:

```
True
False
```

Para o Python, Ana e ana são textos diferentes.

Isso é importante em senhas, nomes de usuários e respostas digitadas.

Atividade 5 — Sistema de senha

Crie um arquivo chamado:

`senha.py`

O programa deve:

1. solicitar uma senha;
2. comparar com a senha correta `senac123`;
3. mostrar `Acesso permitido` ou `Acesso negado`.

Depois teste com:

```
senac123
Senac123
123
```

Responda:

> Por que `senac123` e `Senac123` geram resultados diferentes?

Condições com números

Podemos usar condições para classificar valores.

Exemplo:

```
numero = int(input("Digite um número: "))

if numero > 0:
    print("Número positivo")
else:
    print("Número zero ou negativo")
```

Nesse caso, o `else` agrupa duas possibilidades:

- zero;
- número negativo.

Par ou ímpar

Na Aula 03, vimos o operador %, que mostra o resto da divisão.

Para descobrir se um número é par:

```
numero = int(input("Digite um número: "))

if numero % 2 == 0:
    print("Número par")
else:
    print("Número ímpar")
```

A condição é:

```
numero % 2 == 0
```

Se o resto da divisão por 2 for zero, o número é par.

Atividade 6 — Par ou ímpar

Crie um arquivo chamado:

`par_impar.py`

O programa deve:

1. pedir um número inteiro;
2. verificar se ele é par ou ímpar;
3. mostrar a resposta para o usuário.

Teste com:

- 8;
- 15;
- 0;
- 21.

Depois explique no caderno como o operador % ajudou na solução.

Condições com operadores lógicos

Às vezes, uma única condição não é suficiente.

Exemplo:

Para um aluno ser aprovado, ele pode precisar de:

- média maior ou igual a 7;
- frequência maior ou igual a 75.

Nesse caso, usamos and.

```
if media >= 7 and frequencia >= 75:
    print("Aprovado")
else:
    print("Reprovado")
```

As duas condições precisam ser verdadeiras.

Exemplo com and

```
media = float(input("Digite a média: "))
frequencia = float(input("Digite a frequência: "))

if media >= 7 and frequencia >= 75:
    print("Aprovado")
else:
    print("Reprovado")
```

Teste mentalmente:

Média	Frequência	Resultado
8	90	Aprovado
8	60	Reprovado
6	90	Reprovado
6	60	Reprovado

Atividade 7 — Média e frequência

Crie um arquivo chamado:

`media_frequencia.py`

O programa deve:

1. pedir o nome do aluno;
2. pedir a média;
3. pedir a frequência;
4. aprovar apenas se a média for maior ou igual a 7 e a frequência for maior ou igual a 75.

Exemplo:

```
Nome: João
Média: 8
Frequência: 80
Situação: Aprovado
```

Exemplo com or

O operador `or` significa **ou**.

Exemplo:

Uma loja oferece desconto se o cliente:

- comprar mais de R$ 100,00;
- ou possuir cupom.

```
total = float(input("Digite o total da compra: "))
possui_cupom = input("Possui cupom? s/n: ")

if total > 100 or possui_cupom == "s":
    print("Cliente tem direito ao desconto")
else:
    print("Cliente não tem direito ao desconto")
```

Com `or`, basta uma condição verdadeira.

Atividade 8 — Direito ao desconto

Crie um programa chamado:

`desconto_condicional.py`

O programa deve solicitar:

- nome do cliente;
- total da compra;
- se possui cupom: `s` ou `n`.

Regra:

- se o total for maior que R$ 100,00 **ou** se possui cupom, mostrar que tem direito ao desconto;
- caso contrário, mostrar que não tem direito.

Teste diferentes combinações.

Erros comuns com if e else

Erro de indentação:

```
if idade >= 18:
print("Maior")
```

Erro usando = no lugar de ==:

```
if senha = "1234":
    print("Acesso permitido")
```

Erro esquecendo os dois-pontos:

```
if idade >= 18
    print("Maior")
```

Esses erros são comuns no início e fazem parte do aprendizado.

Teste de mesa com condição

Analise o código:

```
valor = 120
cupom = "n"

if valor > 100 or cupom == "s":
    resultado = "Desconto liberado"
else:
    resultado = "Sem desconto"
```

Complete:

Variável	Valor
`valor`	
`cupom`	
`valor > 100`	

Variável	Valor
`cupom == "s"`	
condição completa	
`resultado`	

Desafio final da aula

Crie um programa chamado:

`caixa_simples.py`

O programa deve solicitar:

- nome do cliente;
- produto;
- preço unitário;
- quantidade;
- forma de pagamento: `dinheiro`, `pix` ou `cartao`.

Regras:

- calcular o subtotal;
- se a forma de pagamento for `pix`, aplicar 5% de desconto;
- caso contrário, não aplicar desconto;
- mostrar subtotal, desconto e total final.

Dica:

```
if forma_pagamento == "pix":
    desconto = subtotal * 5 / 100
else:
    desconto = 0
```

Fechamento

Hoje aprendemos a usar condições para criar decisões.

Trabalhamos com:

- `if`;
- `else`;
- indentação;
- comparação de números;
- comparação de textos;
- operador `%` para par ou ímpar;
- operadores lógicos `and` e `or`;
- testes de mesa com decisões.

Na próxima aula, vamos trabalhar com mais de duas possibilidades usando `elif` e condições encadeadas.